

Take-Home Full-Stack Assessment

Voice Conversation

You are building a browser-based, hands-free voice interface: the user starts a session and then simply talks to an AI assistant that talks back, can be cut off mid-sentence, and plays subtle sounds to signal what it is doing. No phone integration is required; the microphone is the browser's.

The real-time audio loop is the centerpiece. It captures the user's speech, streams it through a pluggable speech-to-text, model, and text-to-speech pipeline, and crucially stops the assistant the instant the user starts talking, then resumes or abandons cleanly. Round-trip responsiveness and natural turn-taking are what we evaluate hardest. Think of the loop and its pluggable pipeline as the reusable core; the browser demo is one way to drive it.

This is a proof of concept, not a production system. We have intentionally left many decisions to you, and how you make them is part of what we are evaluating.

Time expectation: ~5-6 days with effective use of AI coding tools. We do not expect production polish on audio quality; we expect a responsive loop with genuinely good barge-in.

Recommended Stack

Layer	Recommendation
Frontend	React or any React-based framework; browser mic via Web Audio / MediaRecorder
Backend	Node.js or Python, pick one and keep the loop in it
Speech / model	Any STT, LLM, and TTS providers behind your own interface
Infrastructure	Any host that serves over HTTPS (mic access requires it)

What the system does

The voice loop

The browser captures microphone audio and streams it to the backend. Speech-to-text transcribes it, the model produces a reply, and text-to-speech speaks the reply back to the user, all streaming so the assistant can begin speaking before the full reply is generated. A running transcript of both sides is shown as it happens.

Pluggable pipeline

Speech-to-text, the model, and text-to-speech are each swappable behind a clear interface, so a different provider can drop in without touching the loop. Shapes are fixed:

```
interface STT { transcribeStream(audio: AudioStream): AsyncIterable<{ text: string;
final: boolean }>; }
interface LLM { respond(messages: Message[]): AsyncIterable<{ text: string }>; }
interface TTS { synthesizeStream(text: string): AsyncIterable<AudioChunk>; }
```

Barge-in

The defining behavior. The moment the user starts speaking while the assistant is talking, the assistant's audio stops immediately, with no awkward tail. The system then decides what the interruption meant: resume where it left off on "keep going," or treat the new speech as a fresh turn. Latency here is what separates a natural feel from a frustrating one.

Turn-taking and endpointing

The loop detects when the user has finished speaking (end of turn) so it knows when to respond, and handles silence and brief overlap gracefully rather than talking over the user or cutting them off early.

Resume, pause, cancel

"Keep going" resumes a reply that was interrupted. "Hold on" pauses. The user can abandon a reply entirely. Define the behavior and keep it consistent.

Earcons

Short, distinct, non-fatiguing sounds signal state without words: a faint tone when the assistant starts listening, a soft blip when it accepts a request, a gentle chime when something is ready, a descending tone on failure. Each under half a second, injected into the output without clobbering speech.

The dialog interface

So the loop can later be driven by a separate decision engine, the audio bridge talks to a dialog layer over a fixed protocol. Implement the bridge side and a simple stub dialog; the protocol is fixed so a more capable dialog engine could drop in unchanged.

```
// Bridge -> dialog
type FromBridge =
  | { type: "utterance"; text: string; t: number }
  | { type: "pause_detected"; t: number }
  | { type: "interrupt"; t: number };           // user started talking over the
assistant

// Dialog -> bridge
type ToBridge =
  | { type: "say"; text: string }
  | { type: "earcon"; sound: "listening" | "accepted" | "ready" | "failed" }
  | { type: "barge_in"; behavior: "stop" | "pause" | "finish" };
```

Success criteria

These are concrete checks your submission should pass. Some are demonstrable live in the recorded demo; the control-flow ones should be automated against fakes (a scripted STT that emits a known transcript, a canned LLM that returns a known reply, a silent TTS) so they run without live audio or paid providers. State your latency targets in the README and show you meet them.

1. **Barge-in stops immediately.** While the assistant is mid-reply, the user starts speaking. The assistant's audio output halts within your stated target (for example under ~300ms) with no audio tail. Control-flow check: an `interrupt` from the bridge causes TTS emission to stop on the next chunk, not after the current sentence finishes.
2. **Resume an interrupted reply.** After a barge-in interpreted as "keep going," the assistant continues the same reply from where it stopped rather than restarting it. Control-flow check: `barge_in: pause` followed by a resume continues the remaining text.
3. **Fresh turn after interruption.** A barge-in that is genuinely new user speech abandons the prior reply and the new utterance drives the next response. The old reply does not resume.
4. **Endpointing.** After the user stops speaking, the assistant begins responding within your stated window, and it does not cut in during a brief mid-sentence pause. Verifiable with a fake STT that emits a partial, a short gap, then more speech: the assistant waits.
5. **Streaming both ways.** The assistant begins speaking before the full text reply has been generated (TTS streams as the LLM streams), and the transcript updates incrementally as STT partials arrive, rather than appearing all at once at the end.
6. **Earcons at the right moments, non-clobbering.** The listening tone plays when capture starts, and the accepted, ready, and failed sounds play on their events. None of them cut off or garble assistant speech.

7. **Pluggable swap.** Swapping one pipeline component (for example the TTS) for a different implementation behind its interface requires no change to the loop. Demonstrable by running once with a real provider and once with the silent fake.
8. **Graceful awkward cases.** Sustained silence does not trigger a spurious response; the user and assistant starting at the same instant resolves without a deadlock or both talking over each other; a provider hiccup mid-reply surfaces a failed earcon rather than hanging.

Quality bar

The audio bridge must be a standalone module behind the documented dialog protocol, with the pipeline components pluggable behind their interfaces. We should be able to swap the stub dialog for another implementation, or swap a pipeline provider, without rewriting the loop. State your measured round-trip latency target and what you hit.

The demo must work in mobile browsers as well as desktop, served over HTTPS.

Tests where they are meaningful: at minimum, the pipeline interfaces exercised with fakes (a scripted STT, a canned LLM, a silent TTS) to prove the loop's control flow, the barge-in stop, and the endpointing decisions without needing live audio or paid providers in CI.

Graceful handling of the awkward cases: silence, the user and assistant starting at once, a provider hiccup mid-reply.

Thoughtfulness over completeness. A submission with a genuinely snappy barge-in and natural turn-taking beats one with more features that talks over the user or stops with a lag.

Deliverables

- **GitHub repository** with a clear README covering architecture, local setup, which STT / LLM / TTS providers you used and how to configure keys, and how to run the tests. The project must run in GitHub Codespaces.
- **Deployed URL** with the voice demo working end to end over HTTPS, including microphone permission handling.
- **Recorded demo (15-30 minutes)** in two parts. First, the working system: a natural back-and-forth conversation, cutting the assistant off mid-sentence and it stopping instantly, resuming an interrupted reply, the earcons signaling state, and, if feasible, swapping one pipeline component. Second, a detailed architectural walkthrough: talk us through the audio streaming design, the barge-in mechanism and its latency, your endpointing and turn-taking approach, the pluggable pipeline and the dialog protocol, the key tradeoffs you weighed, and what you would change with more time. We want to hear your reasoning, not just see the result.

Evaluation

We weigh, barge-in first and evaluated hardest:

- **Barge-in correctness and latency:** the assistant stops immediately and cleanly when the user speaks
- **Turn-taking and endpointing:** natural, neither talking over nor cutting off the user
- **Pluggable boundaries:** pipeline components and the dialog protocol are honored, swappable without rewrites
- **Audio robustness:** streaming both ways, graceful with silence, overlap, and provider hiccups
- **Transcript clarity:** an accurate running view of both sides
- **Mobile usability** in the browser
- **End-to-end reliability:** working HTTPS deployment, mic permissions handled, no silent failures

The best submissions show real polish and judgment in every detail, above all in how the assistant yields the moment the user speaks.

A quick note on the assessment before you begin. Participation is voluntary and unpaid. It is a screening exercise only, not work for Genius Innovation Lab or for any client, and we do not commercialize your submission or use it in any product or client deliverable. The only exception is that, with your permission, we may share your submission with a prospective client solely so they can evaluate your fit for a role, and never for their own use. You retain full ownership of everything you create and are free to publish it, post it on GitHub, or include it in your portfolio however you wish. Completing the assessment does not create an employment, contractor, or agency relationship, carries no entitlement to compensation, and does not guarantee an offer or placement. You are free to decline or stop at any time by simply replying to let us know.